

Democratic Video Pass-Through for Commercial Virtual Reality Devices

Diego González Morín

Francisco Pereira

Ester González

Pablo Pérez

Alvaro Villegas

Nokia Bell Labs*

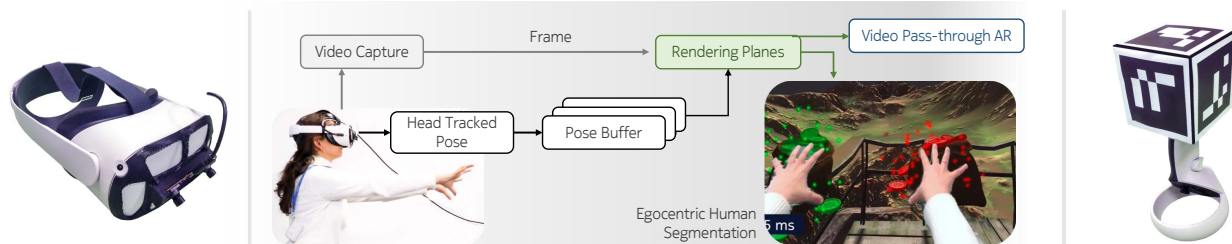


Figure 1: Left) Oculus Quest 2 with our custom hardware for video pass-through. Center) Simplified overview of our software implementation. Right) Custom 3D printed calibration Aruco cube attached to Oculus Quest 2 controller.

ABSTRACT

Video pass-through Extended Reality (XR) is rapidly gaining interest from developers and researchers. However, video pass-through enabled XR devices' ecosystem is still bounded to expensive hardware. In this paper, we describe our custom hardware and software setup for providing effective video pass-through capabilities to inexpensive commercial Virtual Reality (VR) devices. The proposed hardware setup incorporates a low-cost HD stereo camera rigidly hooked to the VR device using a custom 3D printed attachment. Our software solution, implemented in Unity, overcomes hardware-specific limitations, such as cameras' delays, in a simple yet successful manner.

Index Terms: Human-centered computing—Visualization—Video Pass-Through—; Human-centered computing—Virtual Reality—

1 INTRODUCTION

Video pass-through is becoming a key technology enabler in both Augmented Reality (AR) and VR fields. Video pass-through AR overcomes the visual limitations of current AR devices, producing bright and ultra realistic representation of the virtual objects. In VR, video pass-through allows the implementation of Distributed Reality solutions [6], incorporating elements from the user's local reality, such as his/her own hands [4] into the virtual scene. High-end and expensive VR devices, such as the Varjo XR-3, already include a built-in stereo camera for video pass-through immersive applications. In this paper, we briefly describe how to build a custom video pass-through solution for inexpensive commercial devices such as the Oculus Quest 2, which video pass-through solution is limited to low resolution gray scale video. In particular, our contributions are:

- Hardware setup description.
- Camera calibration steps briefing.
- Video frames placement and scaling approach summary.
- Stereo camera capturing delay handling technique description.

2 HARDWARE SETUP

We decided to use the ELP 960P HD OV9750 stereo camera¹ as it provides a wide field of view of 90° and a resolution of 2560x960

* Authors' e-mails: {diego.gonzalez_morin, francisco.pereira_vega, ester.gonzalez, pablo.perez, alvaro.villegas}@nokia-bell-labs.com

¹<https://bit.ly/3qGHY8h>

pixels at a 60 Hz update rate. The camera is attached to the VR goggles using a custom-designed 3D printed attachment as in Fig. 1. We have designed two different attachments: one with the stereo camera's optical axis parallel to the ground, suitable for video pass-through AR, and a second in which the camera's optical axis is tilted 25° towards the ground, optimized for egocentric human segmentation [4] VR applications, see Fig. 1. For each of the described setups, two different attachments have been designed for the Oculus Quest 2 and Pico Neo 3 VR goggles respectively. Our setup is designed to work both in wireless and tethered modes. In the wireless mode, the stereo camera is directly connected to the Thunderbolt connector of the VR goggles. On the second mode, both the stereo camera and the VR goggles are connected to a computer via USB 3.0 ports.

3 SOFTWARE SETUP

Our video pass-through solution, implemented in Unity, seamlessly place the stereo real-time feed within the virtual scene ensuring an accurate alignment between the real and virtual worlds. In any VR applications, the virtual feeds rendered for each eye are captured by two virtual cameras which are translated according to the tracked device's pose within the virtual scene. This process is automatically handled by the VR device. We refer to these cameras as eye cameras, in red in Fig. 2. The captured video frames from the stereo camera, in blue in Fig. 2, are rendered into two virtual planes, which we refer to as rendering planes, one for each eye. These rendering planes, in green in Fig. 2, have to be perfectly placed and scaled relatively to the eye cameras to ensure an accurate alignment between the real world and its virtual representation within the virtual scene. The goal is to perfectly align the rendering planes and stereo cameras coordinate systems. We achieve this in a calibration step, estimating the geometrical relationship $[\Delta x, \Delta y, \beta]$ between the stereo and eye cameras and placing the rendering planes accordingly.

3.1 Video Capture

The video capturing process is the only one which differs between the wireless and tethered modes.

- **Wireless Mode:** in this case the camera is directly connected to the VR goggles. The tested VR goggles, Oculus Quest 2 and Pico Neo 3, are running Android as their operative system. Consequently, we built a native Android plugin to capture the video frames from the stereo camera. The plugin is based in the cross-platform UVC library [1] for webcam video capturing, and is implemented to interface with Unity, allowing the captured frames to be used and rendered in real-time. In

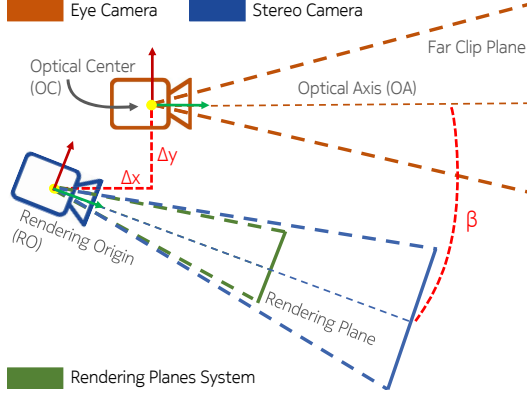


Figure 2: VR device (red), stereo camera (blue) and rendering planes coordinate systems.

each Unity's *Update()* call, a new frame is loaded by Unity from the underlying plugin.

- **Tethered Mode:** in this case, the approach is extremely straight forward: the video frames are directly captured using Unity's scripting API for webcam capturing.

3.2 Rendering Planes Placement

The goals of this step are to correct any distortion produced by the stereo camera lenses, scale $[s_x, s_y]$ the rendering planes, and obtain the geometrical relationship $[\Delta x, \Delta y, \beta]$, as in Fig. 2, between the stereo and eye cameras to properly place the rendering planes relatively to the latter. To correct any distortion produced by the wide field of view camera lenses and estimate the rendering planes' scale, we followed these steps:

1. **Parameters Estimation:** the main goal of this step is to estimate the intrinsic parameters $[f_x, f_y, c_x, c_y]$ and distortion coefficients $[k_1, k_2, k_3, p_1, p_2]$ of our stereo camera. These parameters' description along with the well-known calibration technique we used to obtain them are detailed in [7].
2. **Distortion Correction:** we created a custom shader in Unity to correct the stereo camera's lens distortion using the estimated distortion coefficients and intrinsic parameters. The shader implements the distortion correction equations described in [2]. For each pixel p_{xy} placed in $[x, y]$:

$$x' = x + \bar{x}(k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2\bar{x}^2) + 2p_2\bar{x}\bar{y} \quad (1)$$

$$y' = y + \bar{y}(k_1 r^2 + k_2 r^4 + k_3 r^6) + p_2(r^2 + 2\bar{y}^2) + 2p_1\bar{x}\bar{y}, \quad (2)$$

where $\bar{x} = x - c_x$, $\bar{y} = y - c_y$, $r = \sqrt{\bar{x}^2 + \bar{y}^2}$. The shader is applied to the rendering plane's material so that each new frame is automatically corrected when rendered.

3. **Rendering Plane Scaling:** In this step, we estimate the correct scaling of the rendering planes given an arbitrary distance (d_r) between them and their correspondent eye cameras.

$$S = [s_x, s_y] = [d_r \frac{R_y}{f_y}, d_r \frac{R_x}{f_x}], \quad (3)$$

where s_x, s_y are the rendering plane sizes along each axis and, R_x and R_y each axis' camera resolution.

Estimating the pose of the attached stereo camera relative to the VR goggles' coordinate frame is traditionally done using a camera-IMU (inertial measurement unit) calibration method [5]. However, most of commercial VR devices don't provide access to raw sensor data along with their accurate timestamps, precluding the use of these calibration techniques. We propose a simple yet novel method to estimate the geometry relationship between the stereo camera and the device's coordinate frame. Current VR devices provide hand controllers to interact with the virtual objects. Such controllers' pose is accurately estimated by the VR device relative to its own

coordinate frame. We designed and 3D printed an attachment for both the Oculus Quest 2 and Pico Neo 3 controllers which includes an Aruco cube as the one shown in Fig. 1. Using an standard Aruco tracking library [3], and the camera calibration parameters estimated in previous step, we can accurately track the Aruco cube, and consequently the correspondent controller's pose using the incorporated stereo camera. As the VR devices accurately estimate the pose of their controllers relative to their own coordinate system, we can get the controller's estimated pose relative to both the stereo camera's and VR device's pose within the virtual scene. This information directly gives the geometric relationship $[\Delta x, \Delta y, \beta]$, which we can use to place the rendering frames correctly.

3.3 Delay Alignment Buffers

The stereo camera's image acquisition process consumes processing time (t_c) until the new frame is available in memory. This delay produces the head movement and visual correspondence from the stereo camera feed to be decoupled and unnatural. Consequently, we need to account for such processing time t_c , which we consider to be constant given a VR device, computing platform, and stereo camera model. Instead of setting the rendering planes' origin as child of the eye camera's coordinate frame, we buffer the eye camera's poses and apply the delayed poses, transformed according to the estimated values of $[\Delta x, \Delta y, \beta]$, to the rendering planes' coordinate frame. Once the stereo camera's delay has been experimentally estimated, the buffer's size is automatically calculated. With this simple approach, we manage to accurately align the stereo camera feed with the user's head movement.

4 CONCLUSION

In this paper, we describe our custom inexpensive video pass-through solution for commercial VR devices. The proposed solution barely requires a low-cost HD stereo camera, a rigid 3D-printed attachment and a simple Unity implementation for a correct scaling and positioning of the video pass-through rendering planes. Besides, we include head pose tracking buffers to overcome the stereo camera's capturing latency, ensuring an accurate synchronization between the stereo camera frames and the user's head movement. The implementation described in this paper, provides a simple tool for researchers and developers to explore the usage of video pass-through solutions for immersive experiences.

ACKNOWLEDGMENTS

This work has received funding from the European Union (EU) Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie ETN TeamUp5G, grant agreement No. 813391.

REFERENCES

- [1] libuv: a Cross-Platform Library for USB Video Devices. (2021, Dec).
- [2] D. C. Brown. Close-range camera calibration. *Photogramm. Eng.*, 37, 12 2002.
- [3] S. Garrido-Jurado et al. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6):2280 – 2292, 2014.
- [4] Gonzalez-Sosa et al. Enhanced self-perception in mixed reality: Ego-centric arm segmentation and database with automatic labeling. *IEEE Access*, 8:146887–146900, 2020.
- [5] F. M. Mirzaei and S. I. Roumeliotis. A kalman filter-based algorithm for imu-camera calibration: Observability analysis and performance evaluation. *IEEE Transactions on Robotics*, 24(5):1143–1156, 2008.
- [6] A. Villegas, P. Pérez, and E. González-Sosa. Towards a distributed reality: A multi-video approach to xr. In *Proceedings of the 11th ACM Workshop on Immersive Mixed and Virtual Environment Systems*, p. 34–36, 2019.
- [7] Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.